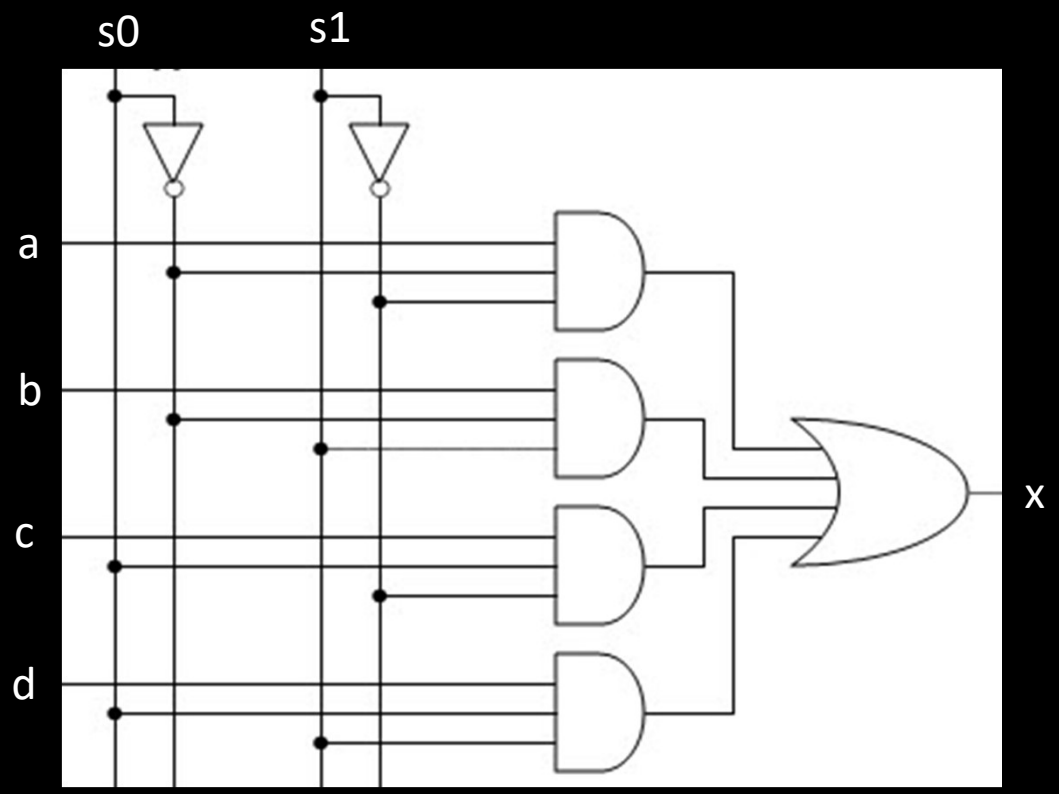
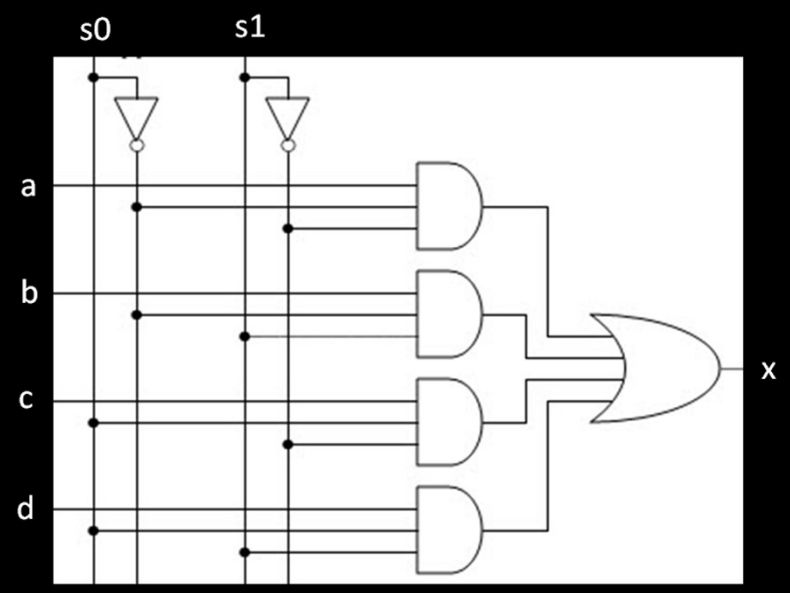
Multiplexador , codificadores e outros Implementações VHDL

Multiplexador 4 x 1 -



Multiplexador 4 x 1

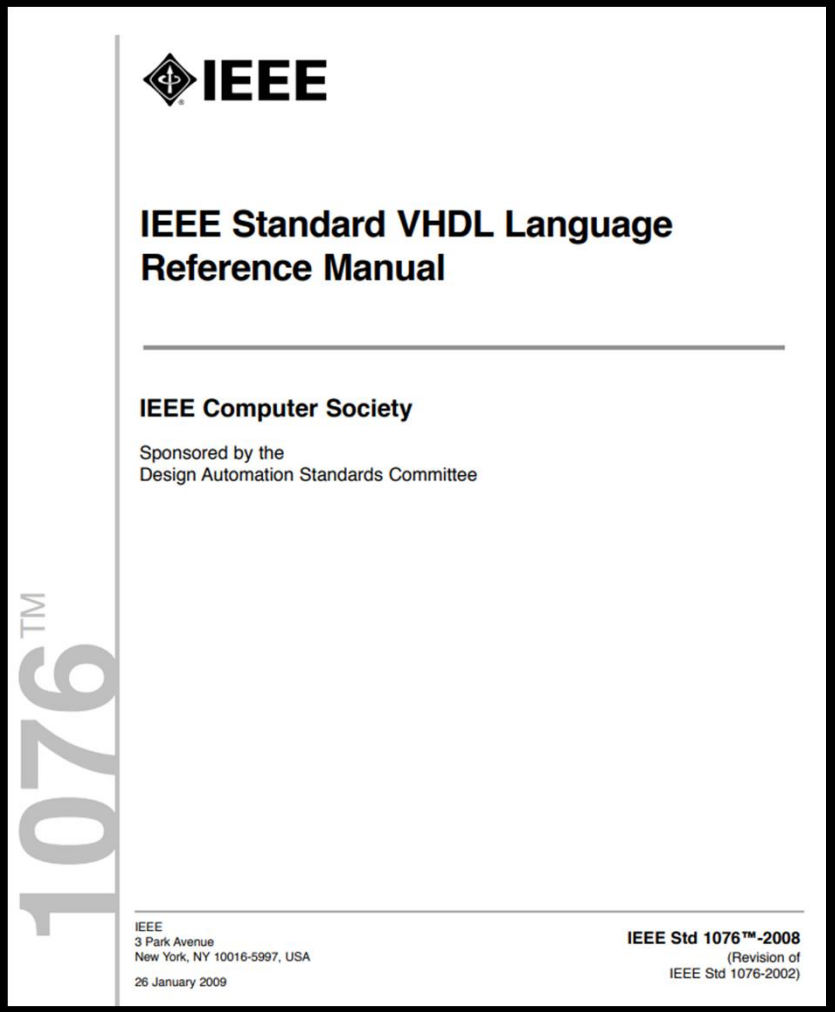


estrutural

```
ARCHITECTURE implementa OF exemplo_mux_1  IS
BEGIN
        x <= (a AND NOT s1 AND NOT s0) OR
             (b AND NOT s1 AND s0) OR
             (c AND s1 AND NOT s0) OR
             (d AND s1 AND s0);
end implementa;
```

Fundamentos de Sistemas Digitais

VDHL Referencia



IEEE

IEEE Standard VHDL Language Reference Manual

IEEE Computer Society

Sponsored by the
Design Automation Standards Committee

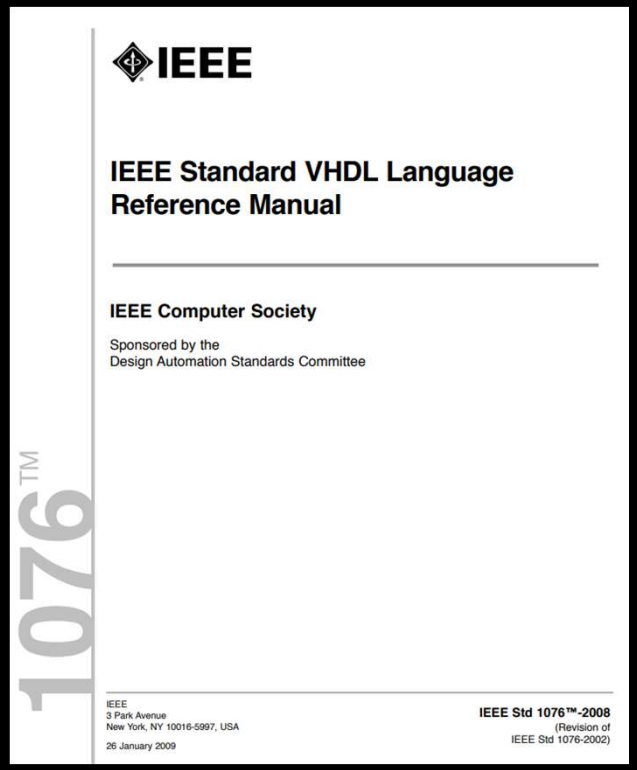
1076™

IEEE
3 Park Avenue
New York, NY 10016-5997, USA

26 January 2009

IEEE Std 1076™-2008
(Revision of
IEEE Std 1076-2002)

VDHL Referencia



IEEE

IEEE Standard VHDL Language Reference Manual

IEEE Computer Society

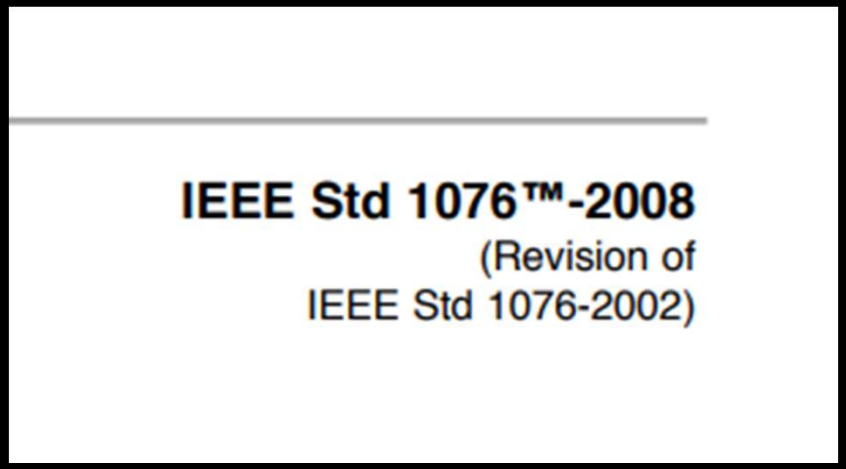
Sponsored by the
Design Automation Standards Committee

1076™

IEEE
3 Park Avenue
New York, NY 10016-5997, USA

26 January 2009

IEEE Std 1076™-2008
(Revision of
IEEE Std 1076-2002)



IEEE Std 1076™-2008
(Revision of
IEEE Std 1076-2002)

Multiplexador 4 x 1 – VDHL estrutura **WHEN ELSE**

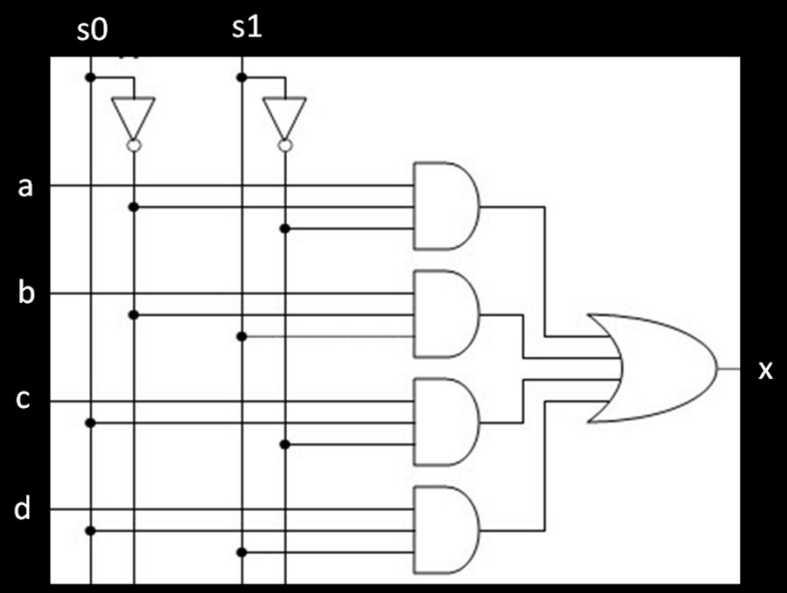
WHEN ELSE permite a transferência condicional de um sinal. Também define uma prioridade na ordem das opções.

Concurrent Statement: Conditional Signal Assignment

The following is an example of a conditional signal assignment:

```
entity control_stmts is
    port (a, b, c: in Boolean;  m: out Boolean);
end control_stmts;

architecture example of control_stmts is
begin
    m <= b when a else  c;
end example;
```

Multiplexador 4 x 1 – implementação 01




```
x <= a when (s1='0' and S0='0') else
     b when (s1='0' and S0='1') else
     c when (s1='1' and S0='0') else
     d when (s1='1' and S0='1');
```

Note: *In IEEE standard 1076-1993, the* ***else*** *clause is optional. If you do not provide an* ***else*** *clause, however, the resulting circuit will probably include a latch, which may not be the desired result.*

Multiplexador 4 x 1

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_4x1_comportamental is
   PORT (
      a, b, c, d, s0, s1: IN STD_LOGIC;
      x: OUT STD_LOGIC
     );
end mux_4x1_comportamental;

architecture Behavioral of mux_4x1_comportamental is
begin

    x <= a when (s1='0' and S0='0') else
         b when (s1='0' and S0='1') else
         c when (s1='1' and S0='0') else
         d when (s1='1' and S0='1');
end Behavioral;
```

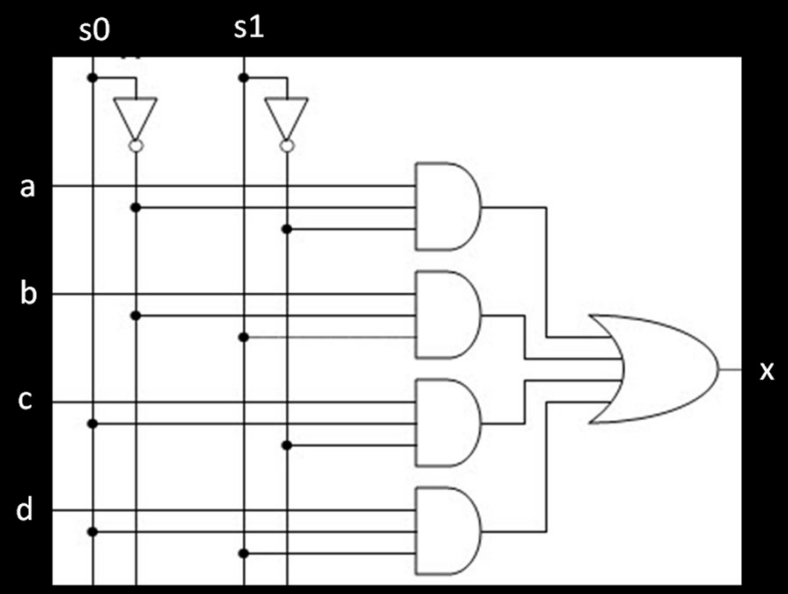
Multiplexador 4 x 1

```
entity mux_4x1_comportamental is
    PORT (
        a, b, c, d, s0, s1: IN STD_LOGIC;
        x: OUT STD_LOGIC
    );
end mux_4x1_comportamental;
```

Multiplexador 4 x 1

```
architecture Behavioral of mux_4x1_comportamental is
begin

    x <= a when (s1='0' and S0='0') else
         b when (s1='0' and S0='1') else
         c when (s1='1' and S0='0') else
         d when (s1='1' and S0='1');
end Behavioral;
```

Multiplexador 4 x 1 – implementação 02




```
process(i0,i1,i2,i3,sel)
 begin
  case sel is
    when "00" => bitout <= i0;
    when "01" => bitout <= i1;
    when "10" => bitout <= i2;
    when others => bitout <= i3;
  end case;
 end process;
```

Multiplexador 4 x 1

```
entity multiplexer4_1 is
port (
        i0 : in std_logic;
        i1 : in std_logic;
        i2 : in std_logic;
        i3 : in std_logic;
        sel : in std_logic_vector(1 downto 0);
        bitout : out std_logic
      );
end multiplexer4_1;
```

Multiplexador 4 x 1

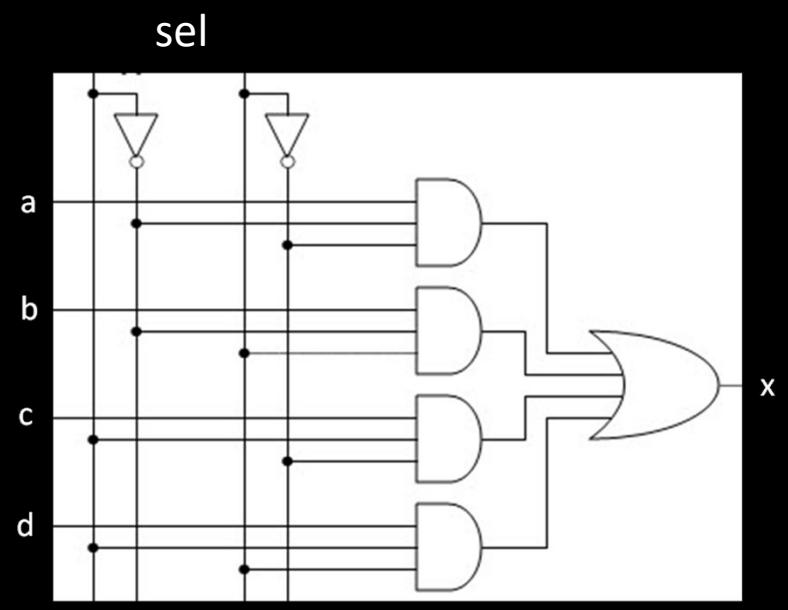
```
architecture Behavioral of multiplexer4_1 is
begin
 process(i0,i1,i2,i3,sel)
  begin
   case sel is
     when "00" => bitout <= i0;
     when "01" => bitout <= i1;
     when "10" => bitout <= i2;
     when others => bitout <= i3;
   end case;
  end process;
end Behavioral;
```

Multiplexador 4 x 1

```
architecture Behavioral of multiplexer4_1 is
begin
 process(i0,i1,i2,i3,sel)
  begin
   case sel is
     when "00" => bitout <= i0;
     when "01" => bitout <= i1;
     when "10" => bitout <= i2;
     when others => bitout <= i3;
   end case;
  end process;
end Behavioral;
```

A selected signal assignment uses the **with** statement, and must include all possible cases. The **others** case ensures that all cases are covered.

Multiplexador 4 x 1 – Implementação 03




```
with sel select
    x <= a when ("00"),
         b when ("01"),
         c when ("10"),
         d when others;
```

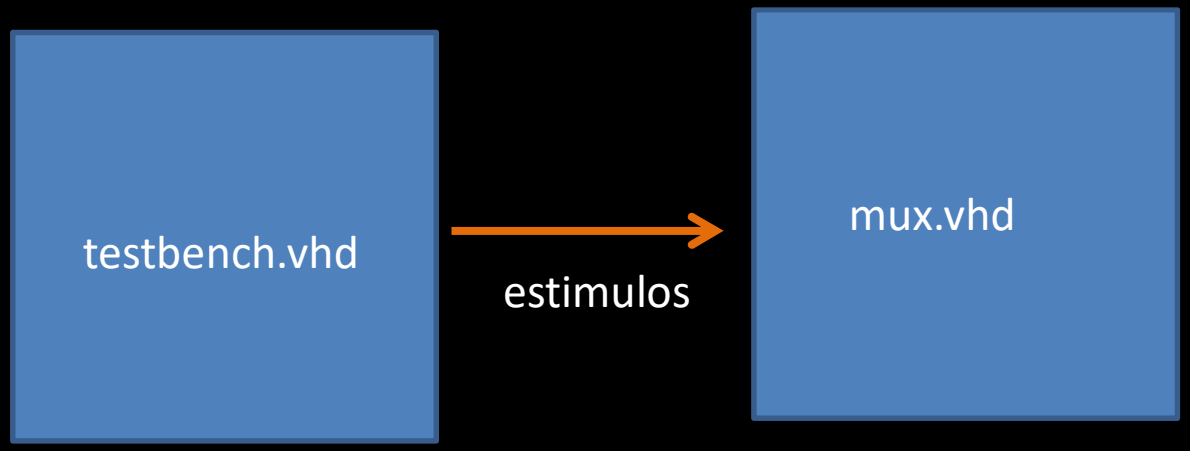
Multiplexador 4 x 1 – Implementação 03

```
entity mux_4x1_comportamental is
    PORT (
        a, b, c, d : IN STD_LOGIC;
        sel : in std_logic_vector(1 downto 0);
        x: OUT STD_LOGIC
      );
end mux_4x1_comportamental;
```

Multiplexador 4 x 1 – Implementação 03

```
architecture Behavioral of mux_4x1_comportamental is
begin
with sel select
    x <= a when ("00"),
         b when ("01"),
         c when ("10"),
         d when others;
end Behavioral;
```

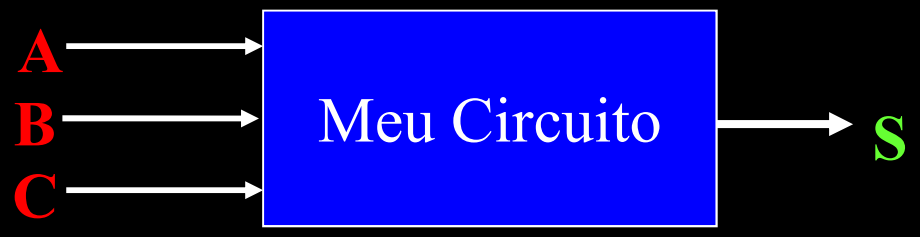
Multiplexador 4 x 1 – TestBench



Descrição de Hardware com HDL – VHDL

Testbench

Descrição em VHDL ou outra linguagem do procedimento de teste para um circuito

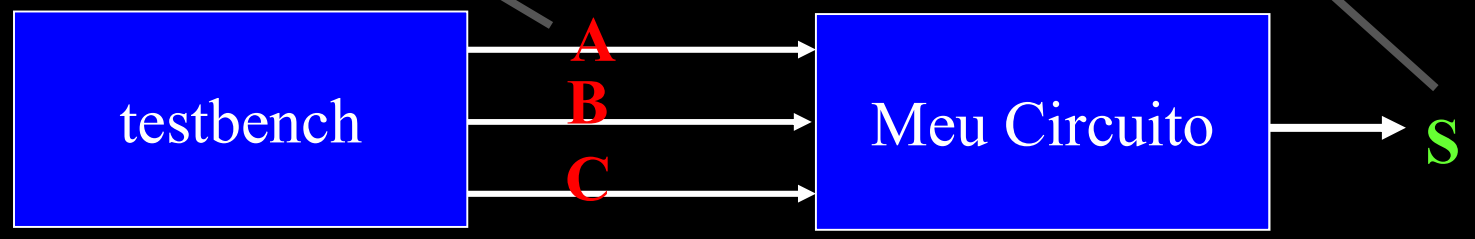


Descrição de Hardware com HDL – VHDL

Testbench

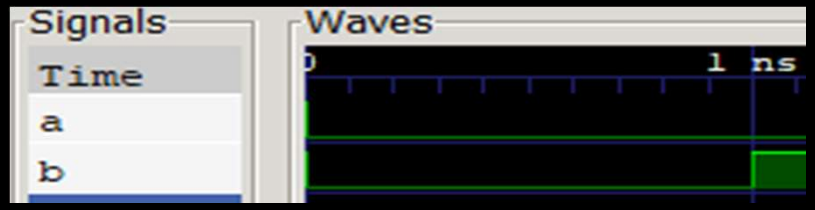
Descrição em VHDL ou outra linguagem do procedimento de teste para um circuito

- Gerar estímulos
- Capturar saídas



Descrição de Hardware com HDL – VHDL Testbench

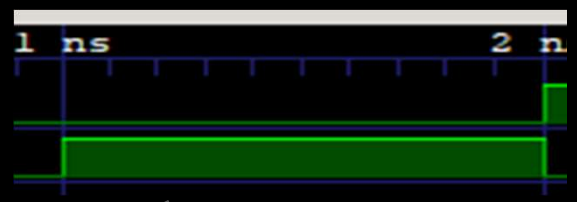
Simulação Bloco OU




```
process
begin
  estimulo_a <= '0';
  estimulo_b <= '0';
  wait for 1 ns;

  estimulo_a <= '0';
  estimulo_b <= '1';
  wait for 1 ns;
```

Descrição de Hardware com HDL – VHDL
Testbench

Simulação Bloco OU




```
process
begin
  estimulo_a <= '0';
  estimulo_b <= '0';
  wait for 1 ns;


  estimulo_a <= '0';
  estimulo_b <= '1';
  wait for 1 ns;
```

Multiplexador 4 x 1 – TestBench – Implementação 01

```
entity testbench is
-- empty
end testbench;

architecture tb of testbench is

signal a, b, c, d, s0, s1 , x: std_logic;
```

Multiplexador 4 x 1 – TestBench – Implementação 01

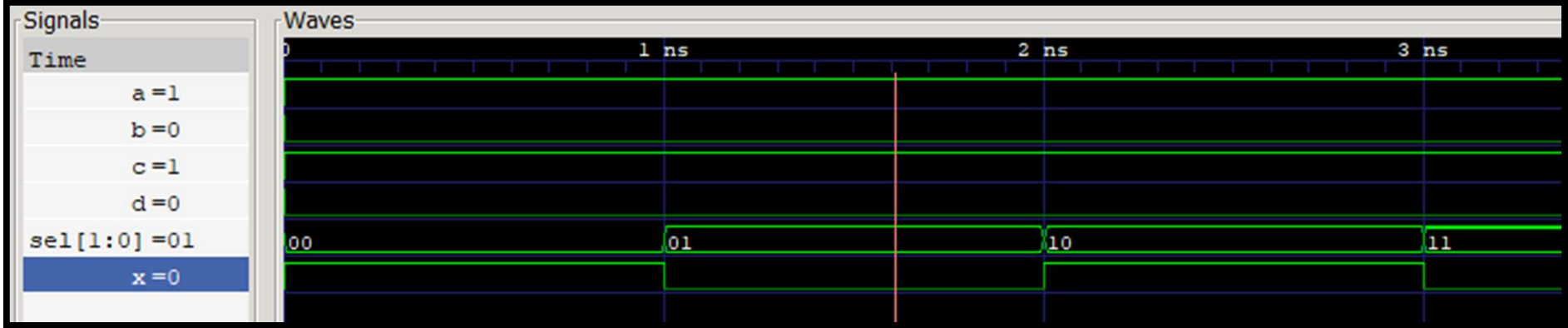
```
UUT : entity work.mux_4x1_comportamental port map (
                                                    a => a,
                                                    b => b,
                                                    c => c,
                                                    d => d,
                                                    s0 => s0,
                                                    s1 => s1,
                                                    x => x
                                                    );
```

Multiplexador 4 x 1 – TestBench – Implementação 01

```
a <= '0';
b <= '1';
c <= '0';
d <= '1';
s0 <= '0';
s1 <= '0';
wait for 1 ns;
```

```
a <= '0';
b <= '1';
c <= '0';
d <= '1';
s0 <= '1';
s1 <= '0';
wait for 1 ns;
```

Multiplexador 4 x 1 – TestBench – Implementação 01



Projeto Circuitos Combinacionais com VHDL Comportamental

```
entity decod_7seg is
port(
  bcd       : IN BIT_VECTOR(3 DOWNTO 0);
  segmentos: OUT BIT_VECTOR(6 DOWNTO 0));
end decod_7seg;

architecture comportamental of decod_7seg is
begin
  WITH bcd SELECT
    segmentos <= "1111110" WHEN "0000",
                 "0110000" WHEN "0001",
                 "1101101" WHEN "0010",
                 "1111001" WHEN "0011",
                 "0110011" WHEN "0100",
                 "1011011" WHEN "0101",
                 "1011111" WHEN "0110",
                 "1110000" WHEN "0111",
                 "1111111" WHEN "1000",
                 "1111011" WHEN "1001",
                 "1111110" WHEN OTHERS;
end comportamental;
```

Projeto Circuitos Combinacionais com VHDL Comportamental

```
entity decod_7seg is
port(
   bcd        : IN BIT_VECTOR(3 DOWNTO 0);
   segmentos: OUT BIT_VECTOR(6 DOWNTO 0));
end decod_7seg;
```

Projeto Circuitos Combinacionais com VHDL Comportamental

```
architecture comportamental of decod_7seg is
begin
  WITH bcd SELECT
    segmentos <= "1111110" WHEN "0000",
                 "0110000" WHEN "0001",
                 "1101101" WHEN "0010",
                 "1111001" WHEN "0011",
                 "0110011" WHEN "0100",
                 "1011011" WHEN "0101",
                 "1011111" WHEN "0110",
                 "1110000" WHEN "0111",
                 "1111111" WHEN "1000",
                 "1111011" WHEN "1001",
                 "1111110" WHEN OTHERS;
end comportamental;
```

Descrição de Hardware com HDL - VHDL

GHDL

GHDL is an open-source simulator for the VHDL language. GHDL allows you to compile and execute your VHDL code directly in your PC.

GHDL runs on Linux, Windows and Apple OS X.

D:\download\ghdl\ghdl_036_mingw32\bin\or_gate.vhd - Notepad++

Arquivo Editar Localizar Visualizar Formatar Linguagem Configurações Tools

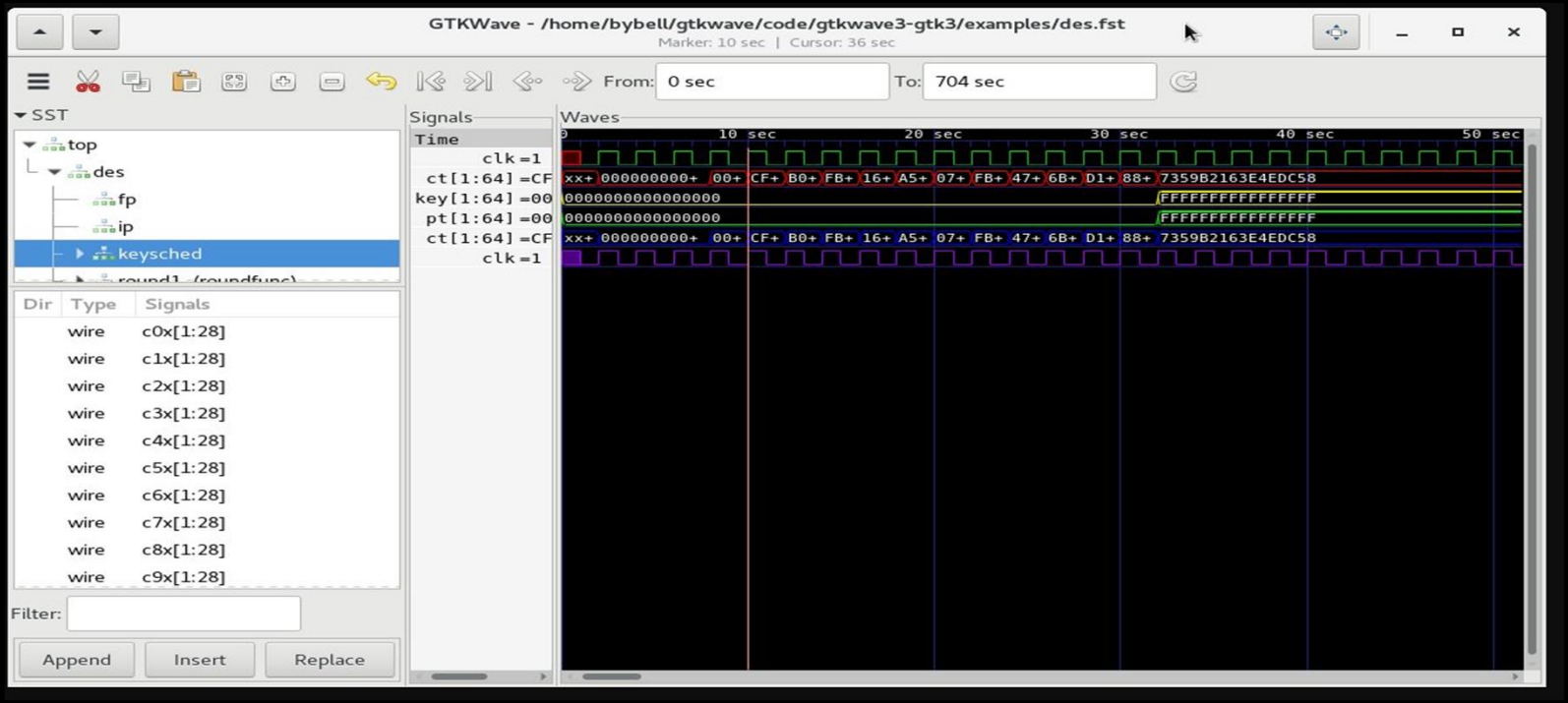
or_gate.vhd | testbench.vhd | half_adder.vhd | half_adder_simple_tb.vhd

```vhdl
-- Simple OR gate design
library IEEE;
use IEEE.std_logic_1164.all;

entity or_gate is
port(
   a: in std_logic;
   b: in std_logic;
   q: out std_logic);
end or_gate;

architecture rtl of or_gate is
begin
   process(a, b) is
   begin
      q <= a or b;
   end process;
end rtl;
```

Descrição de Hardware com HDL - VHDL

GTKWave is an analysis tool used to perform debugging on Verilog or VHDL

Unix and Win32



Descrição de Hardware com HDL - VHDL
GHDL e GTKwave

-a [OPTS] FILEs Analyze FILEs

ghdl -a or_gate.vhd

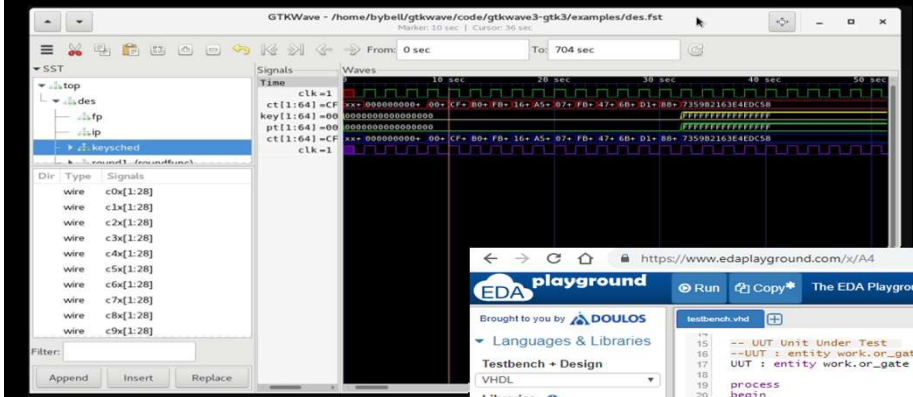
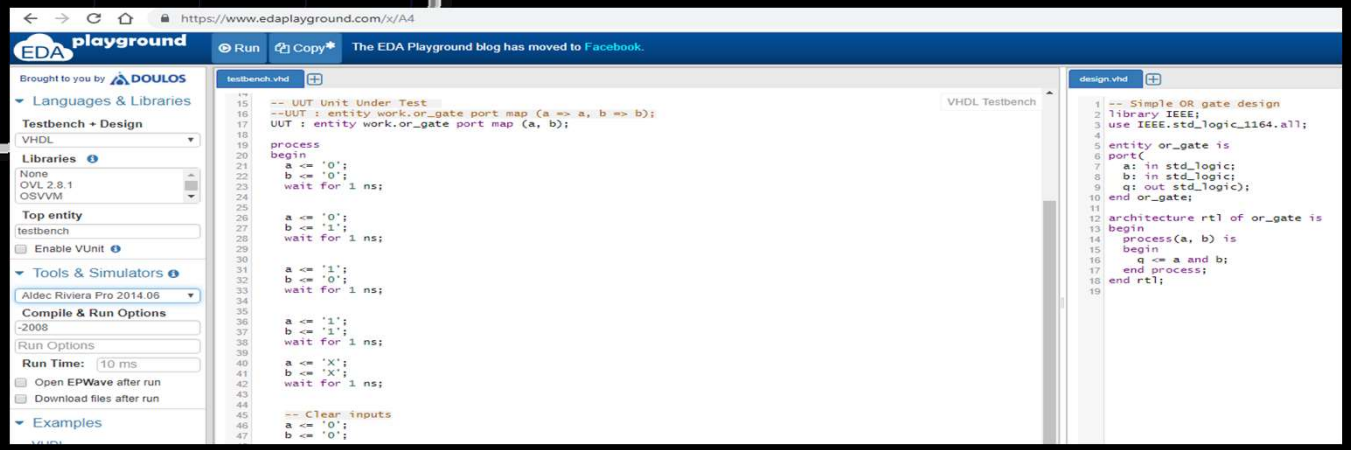
Arquivo VHDL

Descrição de Hardware com HDL - VHDL

```
ghdl -a or_gate.vhd
ghdl -a testbench.vhd

ghdl -e testbench

ghdl -r testbench --vcd=or_gate.vcd

gtkwave or_gate.vcd
```

Descrição de Hardware com HDL - VHDL

Opções desenvolvimento

GHDL

GHDL is an open-source simulator for the VHDL language. GHDL allows you to compile and execute your VHDL code directly in your PC.